



LeetCode 125: Valid Palindrome

1. Problem Title & Link

Title: LeetCode 125: Valid Palindrome

Link: <https://leetcode.com/problems/valid-palindrome/>

2. Problem Statement (Short Summary)

Given a string *s*, determine whether it is a palindrome.

A palindrome reads the same:

Left → Right

Right → Left

Important:

- Ignore spaces
- Ignore special characters
- Ignore punctuation
- Ignore letter case (uppercase/lowercase)

Return:

true

if palindrome, otherwise:

false

Example

Input:

"A man, a plan, a canal: Panama"

After cleaning:

amanaplanacanalpanama

Reverse:

amanaplanacanalpanama

Same.

Output:

true

3. Examples (Input → Output)

Example 1

Input:

s = "A man, a plan, a canal: Panama"

Output:



true

Example 2

Input:

s = "race a car"

Output:

false

Cleaned:

raceacar

Reverse:

racaecar

Different.

Example 3

Input:

s = " "

Output:

true

Explanation:

After removing non-alphanumeric characters:

""

Empty string is considered a palindrome.

Example 4

Input:

s = "Madam"

Output:

true

4. Constraints

$1 \leq s.length \leq 2 * 10^5$

Important:

Ignore non-alphanumeric characters.

Ignore case.

5. Core Concept (Pattern / Topic)

Two Pointers



Secondary Topics:

- String Processing
- Character Validation

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

Clean the string.

Reverse it.

Compare.

Example:

"Madam"

Lowercase:

madam

Reverse:

madam

Same.

Palindrome.

Complexity

Requires:

Extra String

Reversed String

Extra memory used.

Better Observation

To check a palindrome, we only need:

First Character

Last Character

If equal:

Move inward

Repeat.

Example

racecar

Compare:

$r == r$ ✓

Move.



a == a ✓

Move.

c == c ✓

Move.

e

Done.

Palindrome.

Additional Challenge

Input:

"A man, a plan, a canal: Panama"

Need to skip:

Spaces

Commas

Colon

Only compare:

Letters

Digits

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

"A man, a plan, a canal: Panama"

L R

A man, a plan, a canal: Panama

Compare:

A == a

Ignore case.

Move pointers.

Skip:

space

comma

colon

Keep comparing.

Eventually:

All matched

Return:

true

8. Pseudocode (Language Independent)



```
left = 0
right = n-1

while left < right

    skip non-alphanumeric on left

    skip non-alphanumeric on right

    compare lowercase chars

    if different

        return false

    left++
    right--

return true
```

9. Code Implementation

Python

```
class Solution:
    def isPalindrome(self, s: str) -> bool:

        left = 0
        right = len(s) - 1

        while left < right:

            while left < right and not s[left].isalnum():
                left += 1

            while left < right and not s[right].isalnum():
                right -= 1

            if s[left].lower() != s[right].lower():
                return False

            left += 1
            right -= 1

        return True
```

Java

```
class Solution {
```



```
public boolean isPalindrome(String s) {  
  
    int left = 0;  
    int right = s.length() - 1;  
  
    while(left < right) {  
  
        while(left < right &&  
            !Character.isLetterOrDigit(s.charAt(left))) {  
  
            left++;  
        }  
  
        while(left < right &&  
            !Character.isLetterOrDigit(s.charAt(right))) {  
  
            right--;  
        }  
  
        char l =  
            Character.toLowerCase(s.charAt(left));  
  
        char r =  
            Character.toLowerCase(s.charAt(right));  
  
        if(l != r)  
            return false;  
  
        left++;  
        right--;  
    }  
  
    return true;  
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Every character visited at most once.

Space Complexity

$O(1)$

Only pointers used.

11. Common Mistakes / Edge Cases

**Mistake 1**

Not ignoring case.

Example:

Madam

Should be:

true

Mistake 2

Not skipping special characters.

Example:

A man, a plan...

Mistake 3

Using:

== directly

without cleaning.

Edge Cases**Empty String**

Input:

""

Output:

true

Only Symbols

Input:

"!!!"

Output:

true

Single Character

Input:

"a"

Output:

true

12. Detailed Dry Run



Input:

"A man, a plan, a canal: Panama"

Initial:

left = 0

right = last index

Left Char	Right Char	Action
A	a	Match
m	m	Match
a	a	Match
n	n	Match
...	...	Continue

All characters match.

Return:

true

13. Common Use Cases (Real-Life / Interview)

Text Validation

Checking mirrored strings.

DNA Analysis

Sequence symmetry.

Natural Language Processing

Text normalization.

Interview Purpose

Introduces:

Two Pointers

String Processing

14. Common Traps (Important!)

Trap 1

Creating cleaned string first.

Works.

But uses:



$O(n)$
extra space.

Trap 2

Forgetting:
`isalnum()`
check.

Trap 3

Not converting case.
Example:
`A != a`
unless lowercased.

15. Builds To (Related LeetCode Problems)

LC 125 - Valid Palindrome



LC 344 - Reverse String



LC 680 - Valid Palindrome II



LC 5 - Longest Palindromic Substring



LC 647 - Palindromic Substrings

These build palindrome and two-pointer skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Clean + Reverse	$O(n)$	$O(n)$	Good
Stack	$O(n)$	$O(n)$	Fair
Two Pointers	$O(n)$	$O(1)$	Best

Approach 1: Clean and Reverse

`clean = ""`

for `ch` in `s`:



```
if ch.isalnum():  
    clean += ch.lower()
```

```
return clean == clean[::-1]
```

Easy to understand.

Approach 2: Two Pointers (Chosen)

No extra string needed.

Most optimal.

17. Why This Solution Works (Short Intuition)

A palindrome must have matching characters from both ends.

By using two pointers and skipping non-alphanumeric characters, we compare only meaningful characters while moving inward until all pairs are checked.

18. Variations / Follow-Up Questions

Follow-Up 1

Allow deleting one character.

Related:

LC 680

Follow-Up 2

Count all palindromic substrings.

Related:

LC 647

Follow-Up 3

Find longest palindrome.

Related:

LC 5

Follow-Up 4

Check palindrome ignoring only spaces but not punctuation.

Interview Takeaway

This is one of the most important **Two Pointer** problems.

Core pattern:



Compare from both ends

Move inward

Whenever you hear:

Palindrome

Symmetry

Mirror Comparison

immediately think:

Two Pointers

This pattern appears frequently in Zoho, Amazon, Microsoft, and product-company interviews.